

Java 8

The main advantage of Java8 is functional programming.

1. Lambda Expressions
2. Functional Interface
3. Default Methods and Static Methods
4. Predefined Functional Interfaces
 - Predicate
 - Function
 - Consumer
 - Supplier
 - Etc.,
5. Double colon Operator (: :)
 - Method References
 - Constructor References
6. Streams
7. Date and Time API
8. Optional class
9. Nashorn JavaScript Engine

Lambda Expression

The main objective of Lambda expression is to bring benefits of functional programming into java.

Q. What is Lambda Expression?

Ans : It is an anonymous function(nameless function)

- Nameless
- Without return type
- Without modifiers

Ex 1:

```
Public void m1()  
{  
    System.out.println("Hello");  
}
```

```
() → { System.out.println("Hello");  
() → System.out.println("Hello");
```

Ex 2:

```
Public void m2(int a, int b)
{
    System.out.println(a+b);
}
```

(a,b) → { System.out.println(a+b);

Ex 3:

```
Public int squareInt(int n)
{
    return n*n;
}
```

(int n) → {return n*n};

(int n) → n*n; return statement is optional if { } not there

(n) → n*n; type is optional

n → n*n; → () is optional if only one parameter

Ex 4:

```
public void m4(String s)
{
    return s.length();
}
```

S → s.length();

Q. What is Functional Interface?

Ans: An interface that contains only single abstract method then such type of interfaces are called functional interfaces.

Ex:

- **Runnable – run()**
- **Callable – call()**
- **Comparable – compareTo()**
- **Comparator – compare()**
- **ActionListener – actionPerformed()**

Ex:

n → return n*n ; wrong

n → { return n*n ; } correct

n → return n*n} ; wrong

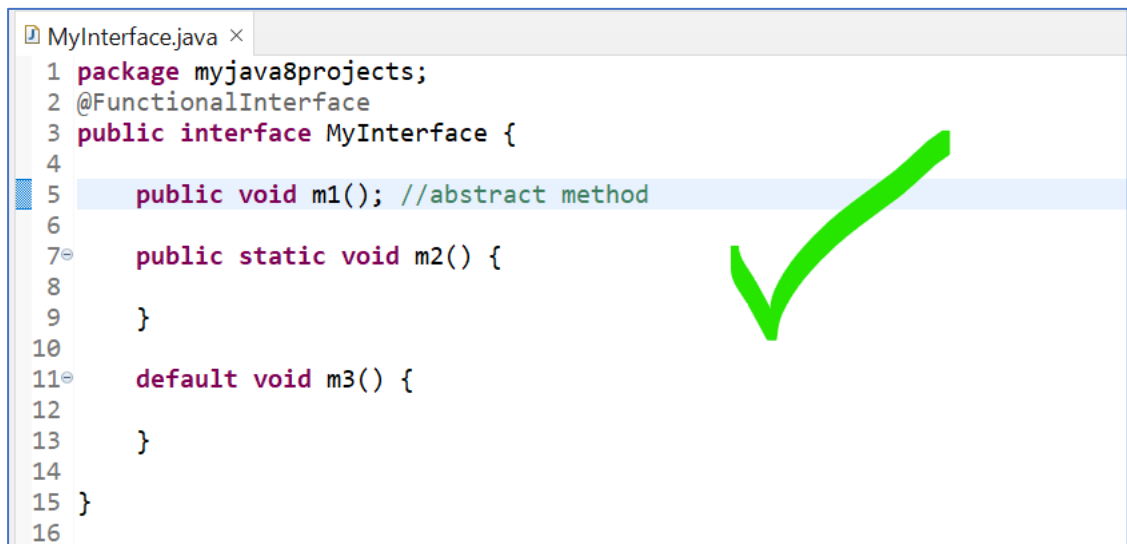
$n \rightarrow \{n*n; \}$; wrong

$n \rightarrow n*n$; correct

- ➔ Without { } we can not use return keyword.
- ➔ Within { } braces if we want to return some value, then compulsory we should use return statement.

Once we write Lambda Expression:

- Functional Interfaces is mandatory to call Lambda Expression
- We can write any number of default and static methods in Functional interface. But there should be only one abstract method.
- @FunctionalInterface can be used to specify an interface is Functional -> **It is optional**
- But when we use @FunctionalInterface annotation then there should be only one abstract method.



```
MyInterface.java x
1 package myjava8projects;
2 @FunctionalInterface
3 public interface MyInterface {
4
5     public void m1(); //abstract method
6
7     public static void m2() {
8
9     }
10
11     default void m3() {
12
13     }
14 }
15 }
16 }
```

Functional Interface with respect to Inheritance

```
MyInterface.java ×
1 package myjava8projects;
2 @FunctionalInterface
3 public interface MyInterface {
4
5     public void m1(); //abstract method
6
7 }
8

B.java × Problems @ Javadoc Declaration Annotations
1 package myjava8projects;
2 @FunctionalInterface
3 public interface B extends MyInterface {
4
5 }
6
```

```
MyInterface.java ×
1 package myjava8projects;
2 @FunctionalInterface
3 public interface MyInterface {
4
5     public void m1(); //abstract method
6
7 }
8

B.java × Problems @ Javadoc Declaration Annotations
1 package myjava8projects;
2 @FunctionalInterface
3 public interface B extends MyInterface {
4
5     public void m1(); //overriding m1 method
6 }
7
```

Interface B contains 2 abstract methods (m1 and m2)

```
MyInterface.java ×
1 package myjava8projects;
2 @FunctionalInterface
3 public interface MyInterface {
4
5     public void m1(); //abstract method
6
7 }
8

B.java × Problems @ Javadoc Declaration Annotations
1 package myjava8projects;
2 @FunctionalInterface
3 public interface B extends MyInterface {
4
5     public void m2(); //overriding m1 method
6 }
7
```



Interface B is normal interface, and it can contain any number of abstract methods

```
MyInterface.java ×
1 package myjava8projects;
2 @FunctionalInterface
3 public interface MyInterface {
4
5     public void m1(); //abstract method
6
7 }
8

B.java × Problems @ Javadoc Declaration Annotations
1 package myjava8projects;
2
3 public interface B extends MyInterface {
4
5     public void m2(); //overriding m1 method
6 }
7
```



Lambda Expression with Functional Interface:

```
public interface InterF{
    public void ml ()
}

public class Sample implements InterF{

    @Override
    public void ml () {
        System.out.println("Hello");
    }
}

public class Test{
    public static void main(String[] args){
        InterF i = new Sample();
        i.ml();
    }
}
```

normal
Code

```
public interface InterF{
    public void ml ()
}

public class Test{
    public static void main(String[] args){
        InterF i = () --> System.out.println("Hello");
        i.ml();
    }
}
```

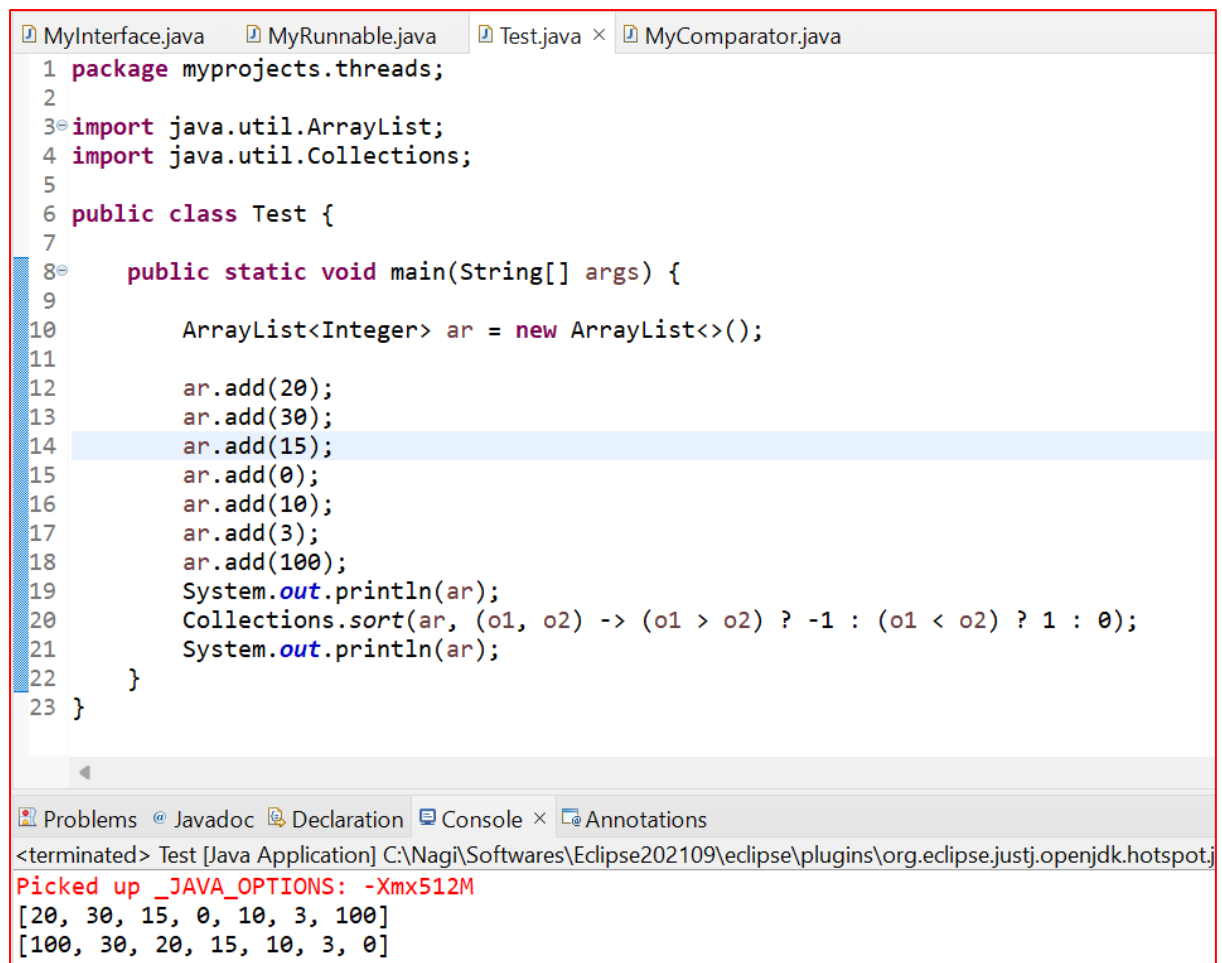
Lambda

Lambda Expressions Multithreading & Collections

Multithreading

```
MyInterface.java  MyRunnable.java  *Test.java x
1 package myprojects.threads;
2
3 public class Test {
4
5     public static void main(String[] args) {
6
7         Runnable r = () -> {
8             for (int j = 0; j < 10; j++) {
9                 System.out.println("Child Thread1");
10            }
11        };
12
13        Thread t = new Thread(r);
14        t.start();
15
16        for (int j = 0; j < 10; j++) {
17            System.out.println("Main Thread1");
18        }
19    }
20 }
21
```

Collections:



```
1 package myprojects.threads;
2
3 import java.util.ArrayList;
4 import java.util.Collections;
5
6 public class Test {
7
8     public static void main(String[] args) {
9
10         ArrayList<Integer> ar = new ArrayList<>();
11
12         ar.add(20);
13         ar.add(30);
14         ar.add(15);
15         ar.add(0);
16         ar.add(10);
17         ar.add(3);
18         ar.add(100);
19         System.out.println(ar);
20         Collections.sort(ar, (o1, o2) -> (o1 > o2) ? -1 : (o1 < o2) ? 1 : 0);
21         System.out.println(ar);
22     }
23 }
```

<terminated> Test [Java Application] C:\Nagi\Softwares\Eclipse202109\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.j

Picked up _JAVA_OPTIONS: -Xmx512M

[20, 30, 15, 0, 10, 3, 100]

[100, 30, 20, 15, 10, 3, 0]

Anonymous Inner class Vs Lambda Expression

```
public class Employee{
    private String name;
    private int eno;

    public void Eployee(String name,int eno){
        this.name= name;
        this.eno = eno;

        public String toString(){
            return eno+"."+ename;
        }
    }
}
```

```

16 class Test
17 {
18     public static void main(String[] args)
19     {
20         ArrayList<Employee> l= new ArrayList<Employee>();
21         l.add(new Employee("Durga",872425));
22         l.add(new Employee("Sunny",212345));
23         l.add(new Employee("Bunny",111213));
24         l.add(new Employee("Chinny",434343));
25         l.add(new Employee("Vinny",424345));
26         System.out.println(l);
27         Collections.sort(l,(e1,e2)->(e1.eno<e2.eno)?-1:(e1.eno>e2.eno)?1:0);
28         System.out.println(l);
29     }
30 }

```

- ➔ Comparator and Comparable interfaces are used to sort collections.
- ➔ Comparator - compare – is used to sort integer numbers
Compare(Object obj1,Object obj2)

It return -ve iff obj1 has to come before obj2
 It return +ve iff obj1 has to come after obj2
 It return 0 iff obj1 and obj2 are equals.

Comparable ➔ compare to used to compare String objects.

```

28
29 Thread t = new Thread()
30 {
31     ... }
32 };
33
34 We are writing a class that extends Thread class
35
36
37 Runnable r= new Runnable()
38 {
39     ... }
40 };

```

ILC

ILC

Subscribe


```
1 class Test
2 {
3     public static void main(String[] args)
4     {
5         Runnable r = new Runnable()
6         {
7             public void run()
8             {
9                 for(int i =0; i<10;i++)
10                {
11                    System.out.println("Child Thread");
12                }
13            }
14        };
15    }
16 }
17 }
18 }
```

- Inner classes are used for instant use.
- Wherever functional interface is there, then we can use Lambda expression.
- Wherever anonymous inner class is there, to resolve complicity we can use Lambda expression

Note: Wherever anonymous inner class is there, then in all scenarios we can't use Lambda expressions. But wherever lambda expression is there, then we can use anonymous inner class. So anonymous inner class most powerful than lambda expression.

- Anonymous inner class can extend a normal class
- Anonymous inner class can extend an abstract class
- Anonymous inner class can implement an interface which contains any number of abstract methods
- Lambda expression can implement an interface which contains a single abstract method(FI).

Default methods & Static methods

- ✓ Still 1.7 Version, every method inside interface by default public & abstract.
- ✓ Every variable inside interface by default public static final.
- ✓ 1.8 V → default methods and static methods
- ✓ 1.9V -> private methods

Note: default methods also known as Virtual Extension methods and Defender methods

- Without any impact on implemented classes, if you want any new method in interface then you should go for **default methods**.

- Default methods can be written in only in interface, but not in class(it treat as switch case default).
- Object class methods can't be defined as default methods in interface as by default Object class methods are available for all classes.

```
interface Left{
    default void m1 () {
        System.out.println("Left interface m1 method");
    }
}

interface Right{
    default void m1 () {
        System.out.println("Right interface m1 method");
    }
}

public class Test implements Left,Right{

    public void m1 () {

        //System.out.println("Test M1 method");
        Right.super.m1 ();
    }

    public static void main(String[] args){

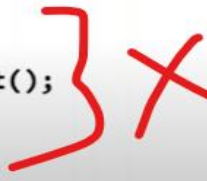
        Test t = new Test ();
        t.m1 ();
    }
}
```

Static methods: Java interface static method helps us in providing security by not allowing implementation classes to override them. We can't define interface static method for Object class methods, we will get compiler error as "This static method cannot hide the instance method from Object.

- To define general utility methods we can go for static methods of Interface.

Note: You must call interface static methods by using interface in implementation class or normal class

```
interface Interf
{
    public static void m1()
    {
        System.out.println("Interface Static Method");
    }
}
class Test implements Interf
{
    public static void main(String[] args)
    {
        Interf.m1();//1 ✓
        m1();//2
        Test.m1();//3
        Test t = new Test();
        t.m1();//4
    }
}
```



- From 1.8 version on-wards we can write static main method inside interface.

Predefined Functional Interfaces

- Predicate
- Function
- Consumer
- Supplier

Two argument predefined functional interfaces

- BiPredicate
- BiFunction
- BiConsumer

Primitive predefined functional interfaces

- IntPredicate
- IntFunction
- IntConsumer,
-

➤ **Predicate(I):**

public abstract boolean test(T t) – wherever some conditional checks are required then we should use predicate functional interface with lambda expressions.

```
1 import java.util.function.*;
2 class Test
3 {
4     public static void main(String[] args)
5     {
6         Predicate<Integer> p1= i->i%2==0;
7         System.out.println(p1.test(10));
8         System.out.println(p1.test(15));
9     }
10 }
11 }
```

→ true
→ false

```

1 import java.util.function.*;
2 class Test
3 {
4     public static void main(String[] args)
5     {
6         String[] s = {"Nag", "Chiranjeevi", "Venkatesh", "Balaiah", "Sunny", "Katrina"};
7         Predicate<String> p = s1 -> s1.length() > 5;
8         for (String s1 : s)
9         {
10             if (p.test(s1))
11             {
12                 System.out.println(s1);
13             }
14         }
15     }
16 }
17 }

```

```

import java.util.function.*;
import java.util.*;
class Employee {
    String name;
    double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }
}

public class Test2 {
    public static void main(String[] args)
    {
        ArrayList<Employee> l = new ArrayList<>();
        l.add(new Employee("Nagi", 1000));
        l.add(new Employee("Siva", 2000));
        l.add(new Employee("Venka", 3000));
        l.add(new Employee("Raj", 4000));
        l.add(new Employee("Naveen", 5000));

        Predicate<Employee> p = e -> e.salary > 3000;
        System.out.println(p);
        for (Employee e1 : l)
            if (p.test(e1)) {
                System.out.println(e1.name + " : " + e1.salary);
            }
    }
}

```

- **Predicate Joining:** Predicate Join can be used to check two predicates.

```

4 public static void main(String[] args)
5 {
6     int[] x = {0,5,10,15,20,25,30,35};
7     Predicate<Integer> p1 = i->i%2==0;
8     Predicate<Integer> p2 = i->i>10;
9     // and(),or(),negate()
10    System.out.println("The numbers which are even and > 10 are:");
11    for(int x1 : x)
12    {
13        if(p1.and(p2).test(x1))
14        {
15            System.out.println(x1);
16        }
17    }
18 }
19 }

```

```

4 public static void main(String[] args)
5 {
6     int[] x = {0,5,10,15,20,25,30,35};
7     Predicate<Integer> p1 = i->i%2==0;
8     Predicate<Integer> p2 = i->i>10;
9     // and(),or(),negate()
10    System.out.println("The numbers which are even or > 10 are:");
11    for(int x1 : x)
12    {
13        if(p1.or(p2).test(x1))
14        {
15            System.out.println(x1);
16        }
17    }
18 }
19 }

```

negate()

```

4 public static void main(String[] args)
5 {
6     int[] x = {0,5,10,15,20,25,30,35};
7     Predicate<Integer> p1 = i->i%2==0;
8     Predicate<Integer> p2 = i->i>10;
9     // and(),or(),negate()
10    System.out.println("The numbers which are not even :");
11    for(int x1 : x)
12    {
13        if(p1.negate().test(x1))
14        {
15            System.out.println(x1);
16        }
17    }
18 }
19 }

```

➤ **Function:**

- ➔ public R apply(T,R);
- ➔ Whenever there is need to perform some operation on the given input and want return some output then we will go for Function functional interface.
- ➔ For Function interface apply method, we have to provide Input type and Return type.

```
1 import java.util.function.*;
2 class Test
3 {
4     public static void main(String[] args)
5     {
6         Function<Integer,Integer> f=i->i*i;
7         System.out.println(f.apply(4));
8         System.out.println(f.apply(5));
9
10
11     }
12 }
```

```
1 import java.util.function.*;
2 class Test
3 {
4     public static void main(String[] args)
5     {
6         Function<String,Integer> f=s->s.length();
7         System.out.println(f.apply("durgasoftwaresolutions"));
8     }
9 }
10
```

```
1 import java.util.function.*;
2 class Test
3 {
4     public static void main(String[] args)
5     {
6         Function<String,String> f=s->s.toUpperCase();
7         System.out.println(f.apply("durgasoftwaresolutions"));
8     }
9 }
10
```

```

8     int marks;
9
10    Student(String name, int marks) {}
11
12
13
14
15    public class StudentGrade {
16    public static void main(String[] args) {
17
18        Function<Student, String> f = s -> {
19            int marks = s.marks;
20            String grade = "";
21            if (marks >= 80)
22                grade = "A";
23            else if (marks >= 60)
24                grade = "B";
25            else if (marks >= 35)
26                grade = "C";
27            else
28                grade = "D - Failed";
29            return grade;
30        };
31
32        Student[] s = { new Student("Nagi", 100),
33                        new Student("Venki", 65),
34                        new Student("Naveen", 45),
35                        new Student("Siva", 25) };
36
37        for (Student s1 : s) {
38
39            System.out.println(s1.name);
40            System.out.println(s1.marks);
41            System.out.println(f.apply(s1));
42            System.out.println();
43        }
44    }
45

```

➤ Function Chaining:

F1.andThen(f2).apply(i) -> first f1 and then followed by f2

F1.compose(f2).apply(i) -> first f2 and then followed by f1

```

import java.util.function.*;

public class TestGrade{
    public static void main(String[] args){

        Function<Integer,Integer> f1 =i -> 2*i;
        Function<Integer,Integer> f2 =i -> i*i*i;
        System.out.println(f1.andThen(f2).apply(2));

    }
}

```



```
import java.util.function.*;

public class TestGrade{
    public static void main(String[] args){

        Function<Integer,Integer> f1 =i -> 2*i;
        Function<Integer,Integer> f2 =i -> i*i*i;
        System.out.println(f1.compose(f2).apply(2));

    }
}
```

➤ **Consumer:**

Public void accept(T); → It takes input and print it. – It won't return anything.

```
};
Consumer<Student> c = s1->{
    System.out.println(s1.name);
    System.out.println(s1.marks);
    System.out.println(f.apply(s1));
    System.out.println();
};

Student[] s = {new Student("Nagi",100),
               new Student("Venki",65),
               new Student("Naveen",45),
               new Student("Siva",25)
};

for(Student s2 :s){
    c.accept(s2);
}
}
```

➤ **Consumer Chaining:**

```

10 class Test
11 {
12     public static void main(String[] args)
13     {
14         Consumer<Movie> c1= m->System.out.println(m.name+" ready to release");
15         Consumer<Movie> c2= m->System.out.println(m.name+" released but it is bigger f
16         Consumer<Movie> c3= m->System.out.println(m.name+" storing information in data
17         Consumer<Movie> cc=c1.andThen(c2).andThen(c3);
18
19         Movie m= new Movie("Spider");
20         cc.accept(m);
21
22     }
23 }
24 }

```

➤ **Supplier:**

Public R get(); → It won't take any input and returns anything.

```

1 import java.util.function.*;
2 import java.util.Date;
3 class Test
4 {
5     public static void main(String[] args)
6     {
7         Supplier<Date> s=()->new Date();
8         System.out.println(s.get());
9         System.out.println(s.get());
10        System.out.println(s.get());
11        System.out.println(s.get());
12    }
13 }
14 }
15 }

```

```

1 import java.util.function.*;
2 import java.util.Date;
3 class Test
4 {
5     public static void main(String[] args)
6     {
7         Supplier<String> s=()->{
8             String otp="";
9             for(int i =0; i<6; i++)
10            {
11                otp=otp+(int)(Math.random()*10);
12            }
13            return otp;
14        };
15
16        System.out.println(s.get());

```

➤ Two input arguments and then perform some operation, then we should go for **BiPredicate**.

```

1 import java.util.function.*;
2 class Test
3 {
4     public static void main(String[] args)
5     {
6         BiPredicate<Integer,Integer> p=(a,b)->(a+b) %2==0;
7         System.out.println(p.test(10,20));
8         System.out.println(p.test(15,20));
9     }
10 }
11

```

Note : Normal predicate can take only one input and perform some conditional check. Sometimes our programming requirement is we must take 2 input arguments and perform some conditional check, for this requirement we should go for BiPredicate.

BiPredicate is exactly same as Predicate except that it will take 2 input arguments.

BiFunction

```

13 class Test
14 {
15     public static void main(String[] args)
16     {
17         ArrayList<Employee> l = new ArrayList<Employee>();
18         BiFunction<Integer,String,Employee> f=(eno,name)->new Employee(eno,name);
19         l.add(f.apply(100,"Durga"));
20         l.add(f.apply(200,"Ravi"));
21         l.add(f.apply(300,"Shiva"));
22         l.add(f.apply(400,"Pavan"));
23         for(Employee e : l)
24         {
25             System.out.println("Employee Number:"+e.eno);
26             System.out.println("Employee Name:"+e.name);
27             System.out.println();
28         }
29     }
30 }

```

BiConsumer

```

16 {
17     ArrayList<Employee> l= new ArrayList<Employee>();
18     populate(l);
19     BiConsumer<Employee,Double> c=(e,d)->e.salary=e.salary+d;
20     for(Employee e:l)
21     {
22         c.accept(e,500.0);
23     }
24     for(Employee e:l)
25     {
26         System.out.println("Employee Name:"+e.name);
27         System.out.println("Employee Salary:"+e.salary);
28         System.out.println();
29     }
30 }
31 }
32 public static void populate(ArrayList<Employee> l)
33 {
34     l.add(new Employee("Durga",1000));
35     l.add(new Employee("Sunny",2000));
36     l.add(new Employee("Bunny",3000));

```

Primitive Version Of Functional Interfaces

```

1 import java.util.function.*;
2 class Test
3 {
4     public static void main(String[] args)
5     {
6         int[] x={0,5,10,15,20,25,30,40};
7         IntPredicate p=i->i%2==0;
8         for(int x1 : x)
9         {
10             if(p.test(x1))
11             {
12                 System.out.println(x1);
13             }
14         }
15     }
16 }
17 }

```

Primitive Predicate Types:

- IntPredicate
- DoublePredicate
- LongPredicate

Primitive Function Types:

There are so many Functions are available. Like,

- **DoubleFunction** -> will take input type as double and return type can be anything.

- **IntFunction** -> will take input type as int and return type can be anything.
- **LongFunction** → will take input type as Long and return type can be anything.
- **DoubleToIntFunction** -> will take input type as double and return type is int
 public int applyAsInt(double d);
- **IntToDoubleFunction**.. etc.
 public double applyAsDouble(int i);
- **ToLongBiFunction**
- **ToLongBiFunction**
- **ToDoubleBiFunction**

Method and Constructor Reference

- Method and Constructor References are an alternative to Lambda expressions.
- An advantage of Method and Constructor References are code reusability – It is an easy form of lambda expression.
- To use method references that method argument must be same.
- If implementation available, then better to use method reference.
- If implementation not available, then better to use Lambda expressions.

```

1 class Test
2 {
3     public static void m1()
4     {
5         for(int i =0; i<10;i++)
6         {
7             System.out.println("Child Thread");
8         }
9     }
10
11     public static void main(String[] args)
12     {
13         Runnable r = Test::m1;
14         Thread t = new Thread(r);
15         t.start();
16         for(int i =0; i<10;i++)
17         {
18             System.out.println("Main Thread");
19         }
20     }
21 }

```



```

1 interface Interf
2 {
3     public void add(int a,int b);
4 }
5 class Test
6 {
7     public static void sum(int x,int y)
8     {
9         System.out.println("The Sum:"+(x+y));
10    }
11    public static void main(String[] args)
12    {
13        Interf i=(a,b)->System.out.println("The Sum:"+(a+b));
14        i.add(10,20);
15
16        Interf i1=Test::sum;
17        i1.add(100,200);
18    }

```

- If the functional interface method returns as object, then we should use constructor reference.

```

1 class Sample
2 {
3     Sample()
4     {
5         System.out.println("Sample class Constructor Execution...");
6     }
7 }
8 interface Interf
9 {
10    public Sample get();
11 }
12 class Test
13 {
14    public static void main(String[] args)
15    {
16        Interf i=Sample::new;
17        Sample s1=i.get();
18    }
19 }

```

```

1 class Sample
2 {
3     Sample(String s)
4     {
5         System.out.println("Sample class Constructor Execution with argument:"+s);
6     }
7 }
8 interface Interf
9 {
10     public Sample get(String s);
11 }
12 class Test
13 {
14     public static void main(String[] args)
15     {
16         Interf i=Sample::new;
17         Sample s1=i.get("Durga");
18     }
19 }
20 }

```

Streams

- If you want to represent a group of objects as single entity, then we should go for Collection.
- If you want process objects from the collection, then we should go java streams.
- Stream is an interface and it is present in **java.util.stream** package.
- Streams can be used for group of elements ,arrays and collections.

filter method(Predicate)

```

1 package myprojects.streams;
2
3 import java.util.Arrays;
4 import java.util.List;
5 import java.util.stream.Collectors;
6
7 public class Test {
8     public static void main(String[] args) {
9
10         List<Integer> al1 = Arrays.asList(2,3,5,10,40);
11
12         al1.stream().filter(i -> i % 2 == 0).collect(Collectors.toList())
13             .forEach(System.out::println);
14     }
15 }
16 }
17

```

map method(Function)


```

1 package myprojects.streams;
2
3 import java.util.Arrays;
4 import java.util.List;
5 import java.util.stream.Collectors;
6
7 public class Test {
8     public static void main(String[] args) {
9
10         List<Integer> marks = Arrays.asList(2,3,5,10,40);
11
12         marks.stream().map(i -> i+5).collect(Collectors.toList())
13             .forEach(System.out::println);
14     }
15 }
16 }
17

```

count method

```

4 {
5     public static void main(String[] args)
6     {
7         ArrayList<Integer> marks = new ArrayList<Integer>();
8         marks.add(70);
9         marks.add(45);
10        marks.add(10);
11        marks.add(65);
12        marks.add(20);
13        marks.add(25);
14        System.out.println(marks);
15        long noOfFailedStudents=marks.stream().filter(m->m<35).count();
16        System.out.println(noOfFailedStudents);
17    }
18 }

```

sorted method

```

1 import java.util.*;
2 import java.util.stream.*;
3 class Test
4 {
5     public static void main(String[] args)
6     {
7         ArrayList<Integer> marks = new ArrayList<Integer>();
8         marks.add(70);
9         marks.add(45);
10        marks.add(10);
11        marks.add(65);
12        marks.add(20);
13        marks.add(25);
14        System.out.println(marks);
15        List<Integer> sortedList=marks.stream().sorted().collect(Collectors.toList());
16        System.out.println(sortedList);
17    }
18 }

```

customized sorting


```

1 package myprojects.streams;
2
3 import java.util.Arrays;
4 import java.util.List;
5
6 public class Test {
7     public static void main(String[] args) {
8
9         List<Integer> marks = Arrays.asList(100,249,5,10,40);
10
11         marks.stream().sorted((i1,i2)->(i1<i2)?1:(i1>i2)?-1:0).forEach(System.out::println);
12     }
13 }
14

```

Default sorting order by using comparable interface

```

MyInterface.java MyRunnable.java Test.java MyComparator.java Employee.java StudentGrade.java
1 package myprojects.streams;
2
3 import java.util.Arrays;
4
5
6 public class Test {
7     public static void main(String[] args) {
8
9         List<Integer> marks = Arrays.asList(100,249,5,10,40);
10
11         marks.stream().sorted((i1,i2)->i2.compareTo(i1)).forEach(System.out::println);
12     }
13 }
14

```

```

1 package myprojects.streams;
2
3 import java.util.Arrays;
4
5
6 public class Test {
7     public static void main(String[] args) {
8
9         List<Integer> marks = Arrays.asList(100,249,5,10,40);
10
11         marks.stream().sorted((i1,i2)->i1.compareTo(i2)).forEach(System.out::println);
12     }
13 }
14

```

Sorting based on length of the string.

```

4 {
5     public static void main(String[] args)
6     {
7         ArrayList<String> l = new ArrayList<String>();
8         l.add("A");
9         l.add("AAAA");
10        l.add("AA");
11        l.add("AAAAA");
12        l.add("AAA");
13        l.add("AAA");
14        System.out.println(l);
15        Comparator<String> c = (s1,s2)->{
16            int l1=s1.length();
17            int l2=s2.length();
18            if(l1<l2) return -1;
19            else if(l1>l2) return +1;
20            else return s1.compareTo(s2);
21        };

```

min and max methods

```
1 import java.util.*;
2 import java.util.stream.*;
3 class Test
4 {
5     public static void main(String[] args)
6     {
7         ArrayList<Integer> l = new ArrayList<Integer>();
8         l.add(10);
9         l.add(0);
10        l.add(15);
11        l.add(5);
12        l.add(20);
13        System.out.println(l);
14        Integer min= l.stream().min((i1,i2)->i1.compareTo(i2)).get();
15        System.out.println(min);
16        Integer max= l.stream().max((i1,i2)->i1.compareTo(i2)).get();
17        System.out.println(max);
18    }
19 }
20 }
```

forEach method: - it accepts Consumer

```
1 import java.util.*;
2 import java.util.stream.*;
3 class Test
4 {
5     public static void main(String[] args)
6     {
7         ArrayList<Integer> l = new ArrayList<Integer>();
8         l.add(10);
9         l.add(0);
10        l.add(15);
11        l.add(5);
12        l.add(20);
13        System.out.println(l);
14        l.stream().forEach(System.out::println);
15    }
16 }
17 }
18 }
```

```
1 import java.util.*;
2 import java.util.stream.*;
3 import java.util.function.*;
4 class Test
5 {
6     public static void main(String[] args)
7     {
8         ArrayList<Integer> l = new ArrayList<Integer>();
9         l.add(10);
10        l.add(0);
11        l.add(15);
12        l.add(5);
13        l.add(20);
14        System.out.println(l);
15        l.stream().forEach(i->{
16            System.out.println("The Square of "+i+" is:"+i*i);
17        });
18    }
19 }
```

Consumer

X	Question	Asker	R...
	l.forEach(i->{ System.out.println(i*i); })	Veeramani Allada	
	already java.util.* is available	kiran kuchimanchi	
	accept()	Sayan Banerjee	
	subpackage classes are not available to parent package	Sayan Banerjee	
	yes sir	kiran kuchimanchi	

already java.util.* is available

Send Privately Send to All

toArray

```
1 import java.util.*;
2 import java.util.stream.*;
3 import java.util.function.*;
4 class Test
5 {
6     public static void main(String[] args)
7     {
8         ArrayList<Integer> l = new ArrayList<Integer>();
9         l.add(10);
10        l.add(0);
11        l.add(15);
12        l.add(5);
13        l.add(20);
14        Integer[] i=l.stream().toArray(Integer[]::new);
15        for(Integer i1 : i)
16        {
17            System.out.println(i1);
18        }
19    }
20 }
```

Getting stream form an array

```

1 import java.util.*;
2 import java.util.stream.*;
3 import java.util.function.*;
4 class Test
5 {
6     public static void main(String[] args)
7     {
8         ArrayList<Integer> l = new ArrayList<Integer>();
9         l.add(10);
10        l.add(0);
11        l.add(15);
12        l.add(5);
13        l.add(20);
14        Integer[] i=l.stream().toArray(Integer[]::new);
15        Stream.of(i).forEach(System.out::println);
16    }
17 }

```

Date and Time API

- Still 1.7V, we have Date, Calendar, TimeStamp... etc., are there.
- Date and Time API also called as JODA Time API → developed by joda.org company/team.

- LocalDate
- LocalTime
- LocalDateTime
- Period

```

import java.time.*;

class TestDate{

    public static void main(String[] args){

        LocalDate localDate = LocalDate.now();

        System.out.println(localDate);

        LocalTime localTime = LocalTime.now();
        System.out.println(localTime);

    }
}

```

```

1 package dateandtime;
2
3 import java.time.LocalDate;
4
5 public class TestDateAndTime {
6     public static void main(String[] args) {
7
8         LocalDate now = LocalDate.now();
9         System.out.println(now);
10
11         int dayOfMonth = now.getDayOfMonth();
12         int monthValue = now.getMonthValue();
13         int year = now.getYear();
14
15         System.out.println(dayOfMonth);
16         System.out.println(monthValue);
17         System.out.println(year);
18
19         System.out.printf("%d/%d/%d", dayOfMonth, monthValue, year);
20     }
21 }

```

```

1 package dateandtime;
2
3 import java.time.LocalTime;
4
5 public class TestDateAndTime {
6     public static void main(String[] args) {
7
8         LocalTime now2 = LocalTime.now();
9
10        System.out.printf("%d:%d:%d:%d", now2.getHour()
11            ,now2.getMinute(),now2.getSecond(),now2.getNano());
12    }
13 }
14

```

```

1 package dateandtime;
2
3 import java.time.LocalDateTime;
4
5 public class TestDateAndTime {
6     public static void main(String[] args) {
7
8         LocalDateTime now = LocalDateTime.now();
9
10        System.out.printf("%d-%d-%d", now.getDayOfMonth(), now.getMonthValue(), now.getYear());
11        System.out.println();
12        System.out.printf("%d:%d:%d:%d", now.getHour(), now.getMinute(), now.getSecond(), now.getNano());
13
14    }
15 }

```

```

1 import java.time.*;
2 public class Test
3 {
4     public static void main(String[] args)
5     {
6         LocalDateTime dt=LocalDateTime.of(1995,Month.MAY,28,12,45);
7         System.out.println(dt);
8     }
9 }
10 }
11

```

```

1 import java.time.*;
2 public class Test
3 {
4     public static void main(String[] args)
5     {
6         LocalDateTime dt=LocalDateTime.of(1995,Month.MAY,28,12,45);
7         System.out.println(dt);
8         System.out.println("After Six Months:"+dt.plusMonths(6));
9         System.out.println("Before Six Months:"+dt.minusMonths(6));
10    }
11 }

```

Period:

```

1 package dateandtime;
2
3 import java.time.LocalDate;
4 import java.time.Period;
5
6 public class TestDateAndTime {
7     public static void main(String[] args) {
8
9         LocalDate birthday = LocalDate.of(1991,6,10);
10        LocalDate now = LocalDate.now();
11        Period p = Period.between(birthday, now);
12        System.out.printf("The Age is %d years %d months %d days", p.getYears(),p.getMonths(),p.getDays());
13
14    }
15 }
16

```

Zoneld

```
1 package dateandtime;
2
3 import java.time.ZoneId;
4 import java.time.ZonedDateTime;
5
6 public class TestDateAndTime {
7     public static void main(String[] args) {
8
9         ZoneId systemDefault = ZoneId.systemDefault();
10        System.out.println(systemDefault);
11
12        ZoneId of = ZoneId.of("America/Los_Angeles");
13        ZonedDateTime zone = ZonedDateTime.now(of);
14
15        System.out.println(zone);
16    }
17 }
18
```

Console ×

<terminated> TestDateAndTime [Java Application] C:\Nagi\Softwares\Eclipse202109\eclipse\plugins\org.ec
Picked up _JAVA_OPTIONS: -Xmx512M
Asia/Calcutta
2023-01-19T02:18:47.952282200-08:00[America/Los_Angeles]